

0.14.1. Deux choses, deux rythmes — de base, ailleurs

Le 0.14 en colonne 10 : la lente et la rapide, au milieu de la ligne.

1. Ce qu'il faut comprendre

C'est le **0.14**, avec un seul changement : la **colonne** des deux `poser` : 10 au lieu de 0.

Les chronomètres ne changent pas : seules les cases où l'on affiche les deux lettres ont bougé.

C'est la **version de base** : une seule chose, à la deuxième place. Le 0.14.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **940 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage – la mise en route de la console
main()         91 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 2      24 o sur place, dans main()
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// _____
// LES VARIABLES : écrites AVANT main(), elles existent pendant
// tout le programme et gardent leur valeur d'un tour à l'autre.
// uint8_t = un nombre entier de 0 à 255.
// _____

#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <poser>    // pose une tuile sur une case du fond
```

2. UNE VARIABLE — LENTE

```
uint8_t lente = 0;           // QUELLE lettre montre la lente : 0 = A, 1 = B ... 25 = Z
```

3. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;          // QUELLE lettre montre la rapide : 0 = A, 1 = B ... 25 = Z
```

4. UNE VARIABLE — IMAGESLENTE

```
uint8_t imagesLente = 0;     // le CHRONOMÈTRE de la lente :  
                             // combien d'images depuis son dernier changement
```

5. UNE VARIABLE — IMAGESRAPIDE

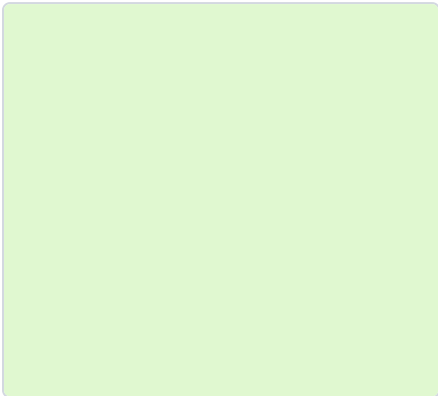
```
uint8_t imagesRapide = 0;    // le CHRONOMÈTRE de la rapide, séparé de l'autre
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

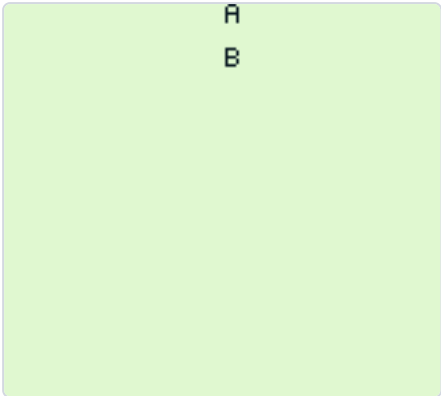
```
int main() {  
  
    // La boucle de jeu : elle ne s'arrête JAMAIS.  
    // Chaque tour dure exactement une image, soit 1/60e de seconde.  
    while (true) {  
  
        image();                // attend l'image suivante (≈ 16,7 ms) :  
                                // c'est ce qui cadence la boucle à 60 tours par seconde  
  
        // À chaque tour, UNE image est passée : les deux chronomètres  
        // avancent d'un cran, en même temps, chacun de son côté.  
        imagesLente++;          // ++ veut dire « ajoute 1 »  
        imagesRapide++;  
  
        // — LA LENTE : elle change toutes les 60 images = toutes les secondes —  
        if (imagesLente == 60) {                                // == veut dire « est égal à ? »  
                                                                // (un seul = voudrait dire « range »)  
            imagesLente = 0;                                     // son chronomètre repart de zéro  
            lente++;                                             // elle passe à la lettre suivante  
            if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :  
                                                                // après Z (25), 26 n'existe pas → retour à A (0)  
        }  
  
        // — LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde —  
        // (60 images ÷ 15 = 4 changements par seconde)  
        if (imagesRapide == 15) {  
            imagesRapide = 0;                                     // son chronomètre repart de zéro  
            rapide++;                                             // elle passe à la lettre suivante  
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A  
        }  
  
        // — L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres —  
        // Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.  
        // Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).  
        poser(10, 0, ALPHABET[lente]); // la lente : colonne 0 → 10  
        poser(10, 2, ALPHABET[rapide]); // la rapide : colonne 0 → 10  
                                         // (la ligne 1 reste vide pour les séparer)  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



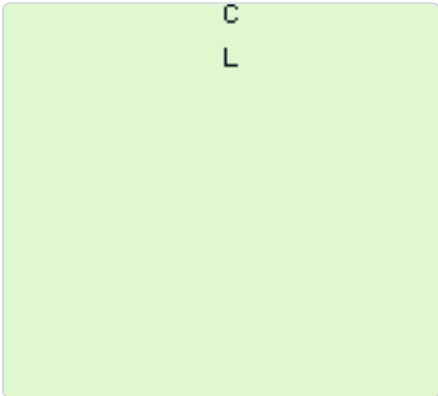
la toute première image — la console vient de démarrer — image 1



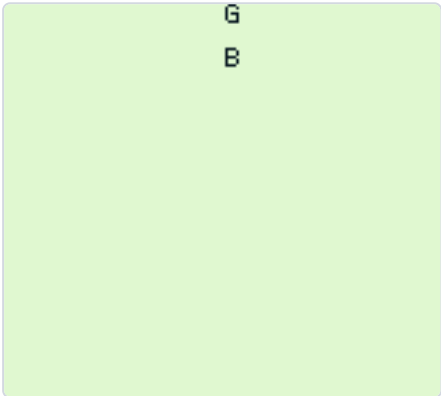
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La lente et la rapide, en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ colonne 10 : la lente (ligne 0) et la rapide (ligne 2) changent

Ensuite : **0.14.2. Deux choses, deux rythmes — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++